

Protocolo de Code Review: prevención de inyección SQL

Destinatarios: desarrolladores junior que revisan pull requests (PR) de backend.

Tiempo estimado por PR: 10-15 minutos si toca la base de datos.

0. Idea clave

Una **inyección SQL** ocurre cuando un dato que escribe el usuario acaba **formando parte de la consulta SQL** en lugar de ser tratado solo como un valor.

sql

-- El código quiere ejecutar esto:

```
SELECT * FROM users WHERE email = 'ana@mail.com'
```

-- Pero si el usuario escribe: ' OR '1'='1

```
SELECT * FROM users WHERE email = " OR '1'='1' -- devuelve TODOS los usuarios
```

Regla de oro: los datos del usuario nunca se pegan dentro del texto de la consulta. Se envían por separado mediante **parámetros**.

Paso 1: Localizar el código que habla con la base de datos

Revisa los archivos modificados del PR y busca todo lo que ejecute SQL. Palabras que debes buscar:

- `execute, query, raw, exec, prepare`
- `createNativeQuery, nativeQuery, FromSqlRaw, sequelize.query, knex.raw`
- Cadenas que contengan `SELECT, INSERT, UPDATE, DELETE, WHERE`

Qué hacer: haz una lista mental o escrita de cada punto donde se ejecuta SQL. Cada uno se revisa individualmente.

Paso 2: Averiguar de dónde vienen los datos

Para cada consulta, pregúntate: ¿de dónde sale cada valor que entra en ella?

Origen

¿Es confiable?

Parámetros de URL, body, formularios, cabeceras, cookies	✗ No
Archivos subidos (CSV, Excel)	✗ No
Datos de APIs externas o webhooks	✗ No
Datos leídos de la propia base de datos (nombres, comentarios)	✗ No (pueden haber sido insertados por un atacante)
Constantes definidas en el código	✓ Sí

Qué hacer: sigue el valor hacia atrás, desde la consulta hasta su origen. Si en algún punto viene de fuera, trátalo como peligroso.

Paso 3: Comprobar cómo se construye la consulta

Esta es la revisión más importante.

🚩 Señales de alarma

Si ves **cualquiera** de estas cosas dentro de una consulta SQL con datos externos, es un hallazgo:

- Concatenación con `+`
- Interpolación: `f"...", `${variable}`, "${variable}"`
- `String.format(...)`, `.format(...)`, `%` de Python
- Construcción de la consulta por trozos (`query += " AND ..."`) con valores del usuario

Ejemplos y corrección

Python

```
python
# ✗ Vulnerable
cursor.execute(f"SELECT * FROM users WHERE email = '{email}'")

# ✓ Corregido
cursor.execute("SELECT * FROM users WHERE email = %s", (email,))
```

Java (JDBC)

```
java
// ✗ Vulnerable
Statement st = conn.createStatement();
st.executeQuery("SELECT * FROM users WHERE id = " + id);
```

//  Corregido

```
PreparedStatement ps = conn.prepareStatement("SELECT * FROM users WHERE id = ?");  
ps.setInt(1, id);  
ps.executeQuery();
```

Node.js

javascript

//  Vulnerable

```
db.query(`SELECT * FROM users WHERE name = '${name}'`);
```

//  Corregido

```
db.query("SELECT * FROM users WHERE name = ?", [name]);
```

Cómo reconocer lo correcto: la consulta es un texto fijo con marcadores (?, %s, :nombre, \$1) y los valores viajan en un argumento aparte.

Paso 4: Revisar el uso del ORM

Usar un ORM (Hibernate, Django ORM, Sequelize, Entity Framework, etc.) **no garantiza seguridad automática**. Protege solo si no se salta sus mecanismos.

Qué buscar:

- Métodos "raw" o consultas nativas (`raw()`, `extra()`, `createNativeQuery`, `FromSqlRaw...`)
- Concatenación dentro de consultas HQL/JPQL
- Filtros creados directamente con datos del cliente, por ejemplo
`User.objects.filter(**request.GET)`

java

//  Vulnerable

```
em.createQuery("FROM User u WHERE u.name = " + name + "");
```

//  Corregido

```
em.createQuery("FROM User u WHERE u.name = :name")  
  .setParameter("name", name);
```

Pregunta de revisión: "¿Es necesario usar una consulta raw aquí? ¿Se puede hacer con el ORM normal?" Si es necesaria, debe estar parametrizada.

Paso 5: Revisar nombres dinámicos (**ORDER BY**, tablas, columnas)

Atención, error muy común: los parámetros (?) **solo sirven para valores**, no para nombres de columnas, tablas ni para **ASC/DESC**. Por eso aquí muchos desarrolladores acaban concatenando.

Señal de alarma: el cliente envía `?sort=nombre` y el código hace `"ORDER BY " + sort`.

Corrección: usar una **lista blanca** (solo se permiten valores que tú defines).

```
python
# ✅ Corregido
COLUMNAS_PERMITIDAS = {"nombre": "name", "fecha": "created_at"}

columna = COLUMNAS_PERMITIDAS.get(sort, "created_at") # valor por defecto si no coincide
orden = "DESC" if direction == "desc" else "ASC"

query = f"SELECT * FROM products ORDER BY {columna} {orden}"
```

Aquí el **f-string** es aceptable porque `columna` y `orden` **solo pueden valer lo que tú has definido**, nunca lo que escribió el usuario.

Paso 6: Revisar casos especiales

LIKE: el comodín va en el valor, no en la consulta.

```
python
# ✅
cursor.execute("SELECT * FROM products WHERE name LIKE %s", (f"%{texto}%",))
```

IN (...): no unir los valores en un texto; crear un marcador por cada elemento.

```
python
# ✅
ids = [1, 2, 3]
marcadores = ",".join(["%s"] * len(ids))
cursor.execute(f"SELECT * FROM users WHERE id IN ({marcadores})", ids)
```

(El **f-string** solo inserta `%s, %s, %s`, nunca los valores.)

LIMIT / OFFSET: convertir a entero y poner un máximo.

```
python
limit = min(int(limit), 100)
```

Paso 7: Revisar procedimientos almacenados y scripts SQL

Un procedimiento almacenado **no es seguro automáticamente**.

Señales de alarma (dentro del código SQL del procedimiento):

- `EXEC(@sql)` o `EXECUTE IMMEDIATE` construyendo texto con parámetros concatenados
- `PREPARE` con concatenación

Corrección: usar parámetros dentro del propio procedimiento (por ejemplo, `sp_executesql` con parámetros en SQL Server).

Paso 8: Revisar validación, errores y permisos

Estos puntos **no sustituyen** a las consultas parametrizadas, pero ayudan.

Validación de entrada

- ¿Un ID numérico se convierte realmente a número antes de usarse?
- ¿Hay límites de longitud y formato razonables?
-  Si ves `replace("'", "''")` o `addslashes()` como protección, es un hallazgo: no es fiable. Pide usar parámetros.

Errores

- ¿Se devuelve al cliente el mensaje de error de la base de datos o la consulta? Debe devolverse un mensaje genérico y registrar el detalle en logs internos.

Permisos

- ¿La aplicación se conecta con un usuario de BD con permisos mínimos? No debería usar `root/sa/admin`.
-

Paso 9: Revisar los tests

- ¿Hay tests para el código que accede a la BD?
 - Para cada corrección de una vulnerabilidad, pide un test que pruebe entradas como:
 - `' OR '1'='1`
 - `'; DROP TABLE users; --`
 - Texto con comillas (`0'Brien`) que debe funcionar sin errores
-

Paso 10: Escribir el comentario en el Code Review

Un buen comentario indica **dónde**, **por qué** y **cómo** corregirlo.

● Bloqueante: posible inyección SQL

En `UserRepository.java`, línea 42, el parámetro `name` se concatena en la consulta. Un usuario podría manipular la consulta.

Corrección sugerida: usar `PreparedStatement` con `?` y `ps.setString(1, name)`.

Cómo priorizar:

Nivel	Cuándo	Acción
● Bloqueante	Dato externo concatenado en SQL	No aprobar el PR
● Importante	<code>ORDER BY</code> dinámico sin lista blanca; raw query sin justificar	Pedir cambio antes de aprobar
● Mejora	Permisos excesivos, falta de tests, validación débil	Sugerir; puede ir en otro PR

Ante la duda, **pregunta o escala a alguien con más experiencia en seguridad**. Es mejor preguntar que aprobar con dudas.

✅ Checklist final del revisor

- He localizado todas las consultas SQL del PR
- He identificado de dónde viene cada valor
- No hay concatenación ni interpolación de datos externos en SQL
- Se usan parámetros (`?`, `%s`, `:nombre`)
- Los métodos raw/nativos del ORM están justificados y parametrizados
- `ORDER BY`, tablas y columnas dinámicas usan lista blanca
- `LIKE`, `IN` y `LIMIT` están bien tratados
- Los procedimientos almacenados no concatenan SQL
- No se exponen errores de BD al cliente
- La cuenta de BD no tiene más permisos de los necesarios
- Hay tests de los casos peligrosos
- He dejado comentarios claros con la corrección propuesta

⚠ Falsas seguridades

Creencia	Realidad
"Uso un ORM, no hay riesgo"	Los métodos raw y los filtros dinámicos siguen siendo vulnerables
"Valido la entrada en el frontend"	Un atacante puede saltarse el frontend
"Escapo las comillas a mano"	Es frágil y fácil de evadir
"Es un endpoint interno"	Los datos internos también pueden venir manipulados
"Tenemos WAF"	Es una capa extra, no sustituye a corregir el código

Si quieres, puedo adaptar este protocolo a vuestro stack concreto (Django, Spring, Express, .NET...) o prepararlo como documento descargable (Word o PDF) para el equipo.